



Society of Petroleum Engineers

SPE-229206-MS

AI-Driven Optimization of Chemical Consumption in Oil Pipelines

Arman Bari, Waqas Ahmed, Malik Waqas, Saad Mansoor, Asad Hayat Akhter, and Muhammad Zohaib Hassan, PAK-ARAB REFINERY, Karachi, Sindh, Pakistan; Abdul Sami, Chemical Engineering Department, University Teknologi PETRONAS, Perak, Malaysia

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/229206-MS](https://doi.org/10.2118/229206-MS)

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

The use of Drag Reducing Agents (DRAs) in crude oil pipeline transportation is critical for enhancing flow efficiency and minimizing pressure losses. However, traditional static dosing strategies often result in chemical overuse or underperformance leading to increased costs and operational inefficiencies. This study presents a novel AI-driven framework to optimize DRA injection rates using supervised machine learning models trained on real-world operational data. Key parameters such as crude flowrate, suction and discharge pressures, valve positions, crude oil density and pumping configurations were analyzed over an eight-month period. A comparative evaluation of five regression models Decision Tree, Random Forest, Support Vector Regression, K-Nearest Neighbors and XGBoost was conducted. Among them, the Random Forest model demonstrated superior performance, achieving the highest predictive accuracy ($R^2 = 0.80$, $RMSE = 0.51$). The framework not only reduced DRA consumption without compromising throughput but also standardized decision-making across operational shifts. This research highlights the transformative potential of machine learning in pipeline operations enabling cost-effective, sustainable and data-driven DRA management in the oil and gas industry.

Introduction

Oil pipelines are fundamental to the global energy infrastructure enabling the large-scale, continuous and cost-effective transportation of crude oil and refined petroleum products across regions and borders. In recent years, as the oil and gas sector faces increasing pressure to enhance operational efficiency, reduce energy consumption, and meet stringent environmental standards, improving the internal flow dynamics of pipeline systems has emerged as a critical engineering objective.

One of the most widely adopted techniques to enhance hydraulic efficiency in pipelines is the use of Drag Reducing Agents (DRAs) high molecular weight polymeric additives that reduce turbulent frictional losses in the fluid, thereby lowering pressure drops and increasing throughput. The first large-scale deployment of DRA technology occurred in the Trans-Alaskan Pipeline in 1979, where a 50% drag reduction increased pipeline capacity from 1.45 to 2.1 million barrels per day, eliminating the need for two additional pumping stations (Burger, 1982; Jubran et al., 2005). Since then, DRA implementation has expanded across major

pipeline systems including the Iraq–Turkey crude oil pipeline and the North Sea's Oseberg Field (Berge, 1996; Jubran et al., 2005)

DRA offer several advantages: increased pipeline capacity, reduced pumping energy, smaller pipe diameters in new designs, and reduced pressure surge risks all contributing to overall cost savings and improved system flexibility (Jubran et al., 2005). Dosage levels for DRAs typically range from 1–2 ppm for stabilized single-phase flows but may be up to four times higher in multiphase systems due to elevated shear-induced degradation (Berge, 1996).

Despite their effectiveness, DRA injection strategies remain largely rule-based in real-world operations. Fixed dosing schedules determined by historical experience or conservative assumptions often fail to respond to dynamically changing conditions, such as fluctuations in crude type (light vs. heavy), pipeline segment hydraulics, temperature, flow rate, and valve positions. This can lead to overdosing, increasing chemical costs and environmental impact, or underdosing, resulting in suboptimal drag reduction and reduced throughput.

In an effort to address these limitations, earlier studies have explored algorithmic solutions for DRA optimization using Dynamic Programming (DP). One such approach involves a limited exhaustive search over discrete DRA injection levels (e.g., 5, 10, 15, 20 ppm) per pipeline batch and evaluates the trade-off between DRA consumption and energy cost by simulating downstream effects using a flow optimization model (Modisette, n.d.). These models simulate DRA degradation (e.g., 1% per mile) and apply heuristic rules such as temporarily shutting down centrifugal pumps to prevent DRA destruction, leading to a batch-wise optimal injection plan. While effective in structured simulations, such methods are limited by their reliance on static rules, fixed dosage intervals, and the absence of real-time feedback loops.

In contrast, this study proposes a novel AI-driven framework that leverages machine learning and historical operational data to dynamically optimize DRA injection rates in real-time. By analyzing pipeline behavior across parameters such as pressure, crude oil grade, valve opening percentage, production scheduling needs, and temperature, the model learns optimal dosing patterns under varying operational conditions. Unlike DP-based static optimization, this approach continuously adapts to changing flow conditions and improves over time through predictive learning.

The goal of this research is to design, implement, and validate a machine learning-based DRA optimization system capable of minimizing chemical usage without compromising hydraulic efficiency. The framework is tested on historical operational datasets from real pipeline operations and demonstrates substantial improvements in dosing accuracy, chemical savings, and energy consumption. It offers a scalable solution for smart pipeline management and contributes to the broader industry shift toward sustainable, intelligent automation in oil transportation.

Methodology

This section outlines the methods used to develop an AI-driven predictive model for optimizing chemical consumption in oil pipelines. Best historical data were utilized to train five different regression models: Decision Tree, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and XGBoost. The objective was to accurately predict the optimal injection rate of drag-reducing agents based on the input features. The development of the predictive model, using any of the aforementioned algorithms, can be divided into four key stages and each step is explained compressively in the following sub-sections.

1. Data collection and preprocessing,
2. Model formation,
3. Training,
4. Performance testing and evaluation

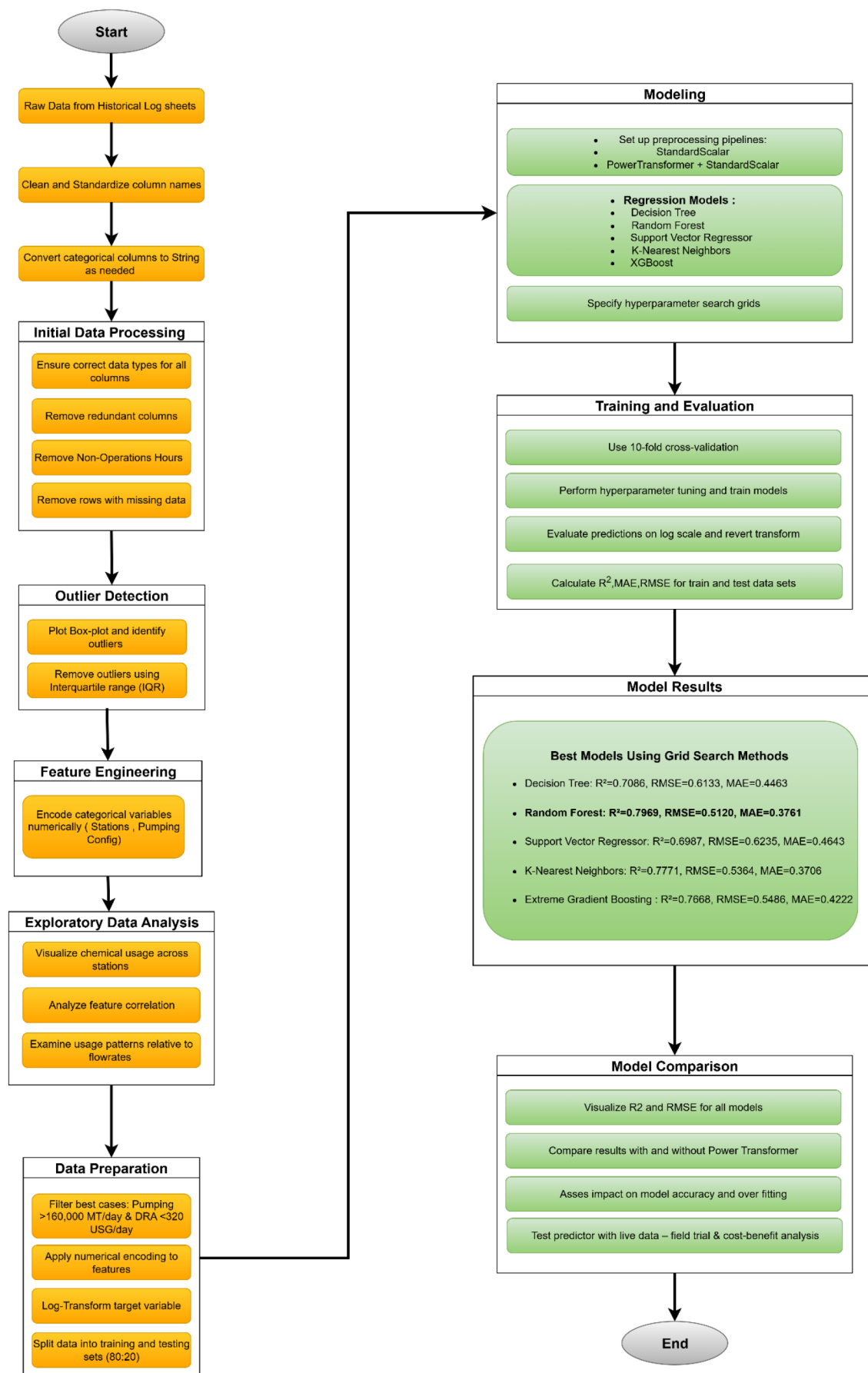


Figure 1—Modeling Framework adopted in this work

Data Collection and Preprocessing

Data Source

The dataset used in this study was obtained from hourly Drag Reducing Agent (DRA) injection log sheets collected over an eight-month operational period. These records provide a detailed account of pipeline operating conditions and chemical dosing practices, offering a rich source of information directly related to DRA consumption in long-distance crude oil pipelines.

Key operational features captured in the dataset include flowrate ($\text{MT}\cdot\text{h}^{-1}$), suction and discharge pressures ($\text{kg}\cdot\text{cm}^{-2}$) at each pumping station, pressure control valve (PCV) opening (%), and pumping configurations that specify the number of active pumps across the four stations (e.g., (2+2+2+2), (1+0+1+0), (2+0+1+1), (2+1+0+1)). Additional variables include crude oil density ($\text{g}\cdot\text{cm}^{-3}$) at each station and the required throughput ($\text{MT}\cdot\text{h}^{-1}$), which reflects production targets under specific operational constraints. The DRA injection rate ($\text{gal}\cdot\text{h}^{-1}$) is designated as the target variable for optimization, with the goal of minimizing energy consumption while maintaining or enhancing throughput.

To ensure the model learns from most efficient operational conditions, training was conducted using a filtered subset of best instances defined as observation with high throughput ($>16000 \text{ MT/Day}$) and low DRA consumption, under valid and stable conditions. By focusing on these optimal cases, the model was able to learn efficient injection patterns while avoiding bias from suboptimal or noisy data. This selective training approach ensured that the resulting model prioritized operational efficiency and chemical cost savings.

Exploratory Data Analysis

An initial exploratory analysis was performed to detect overdose/underdose regimes of DRA consumption. [Figure 2](#) (Monthly trend DRA vs Crude Throughput) illustrates a representative overdose episode during July, where increasing the dose from ~ 360 to $\sim 420 \text{ gal}\cdot\text{h}^{-1}$ yielded no material improvement in crude flow capacity, thereby unnecessarily inflating chemical costs. This observation motivated the construction of a supervised learning model to recommend an optimized injection rate conditional on operating conditions (desired flowrate, density, pressures, station PCV position, pumping configuration, and required throughput)

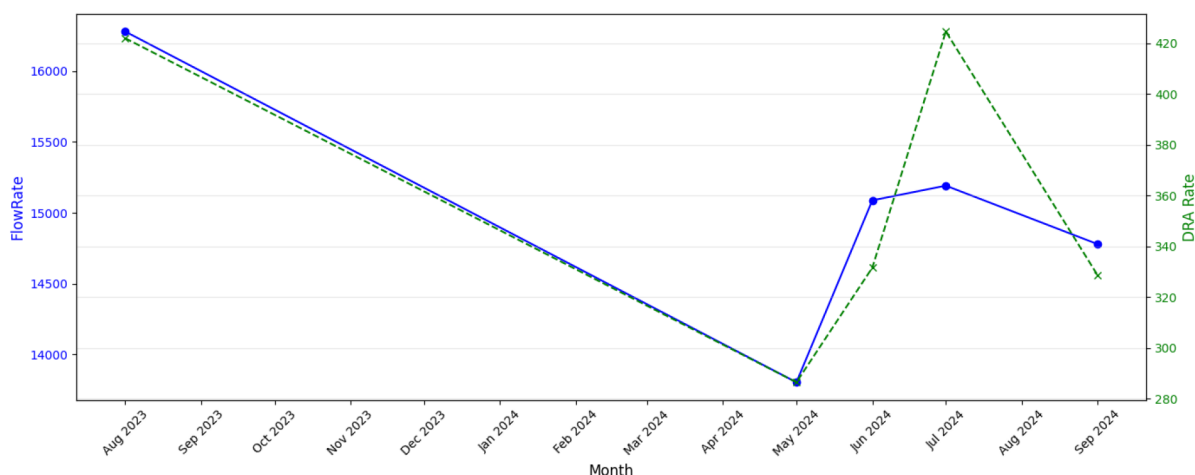


Figure 2—Monthly Trend DRA vs Crude throughput

Data Preprocessing

The dataset underwent a series of preprocessing steps to ensure quality and reliability before being used for machine learning models. Raw operational logs often contain missing entries, anomalous readings, and

non-operational data due to manual input errors, instrument malfunctions, or shutdown periods. Therefore, a systematic data-cleaning and preparation pipeline was implemented.

Handling Missing Data

The pumping configuration variable, which is manually recorded by operators, contained missing values and inconsistent entries (e.g., special characters). To address this, missing entries were first identified and treated as NULL. A forward-fill imputation method was then applied to propagate the last valid recorded configuration forward. This imputed dataset was reviewed and validated by data custodians to ensure accuracy and operational consistency.

Removal of Non-Operational Data

Records corresponding to pipeline shutdowns (i.e., configurations such as (0+0+0+0)) were excluded from the modeling dataset. These entries typically represent refinery turnarounds or feed tank filling operations, which do not reflect normal DRA usage patterns.

Data Type and Consistency Checks

All variables were converted to the correct data types (numeric or categorical) and verified for logical consistency. Invalid records or garbage data entries, such as negative pressures or flowrates, were removed.

Outlier Detection and Removal

To ensure the reliability of the predictive modeling process, a comprehensive outlier detection analysis was undertaken as part of the data preprocessing workflow. Outliers, if left untreated, can introduce significant bias, distort statistical properties, and degrade the performance of machine learning algorithms. However, in industrial operations, extreme values may also represent critical events or unique operating conditions rather than errors. Therefore, a rigorous review process was adopted to distinguish between erroneous readings and meaningful operational anomalies.

A statistical approach based on the Interquartile Range (IQR) was employed to systematically identify potential anomalies across six key numerical features:

- Crude flowrate ($\text{MT} \cdot \text{h}^{-1}$)
- DRA injection rate ($\text{gal} \cdot \text{h}^{-1}$)
- Crude density ($\text{g} \cdot \text{cm}^3$)
- Suction pressure ($\text{kg} \cdot \text{cm}^2$)
- Discharge pressure ($\text{kg} \cdot \text{cm}^2$)
- PCV opening (%)

To facilitate the detection of outliers, box plots were constructed using the Seaborn and Matplotlib libraries. These visualizations provide a concise summary of data distribution, highlighting values that fall beyond $1.5 \times \text{IQR}$ below the first quartile (Q1) or above the third quartile (Q3).

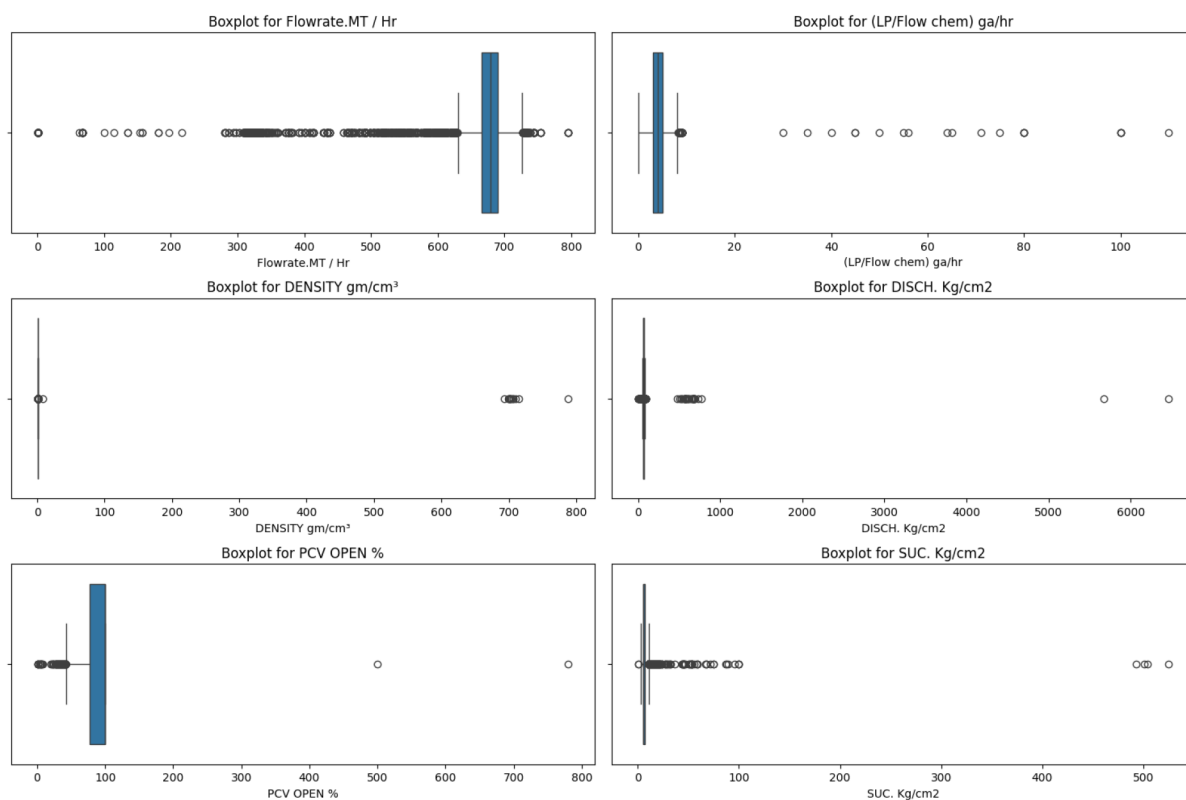


Figure 3—Box plots of key process variables

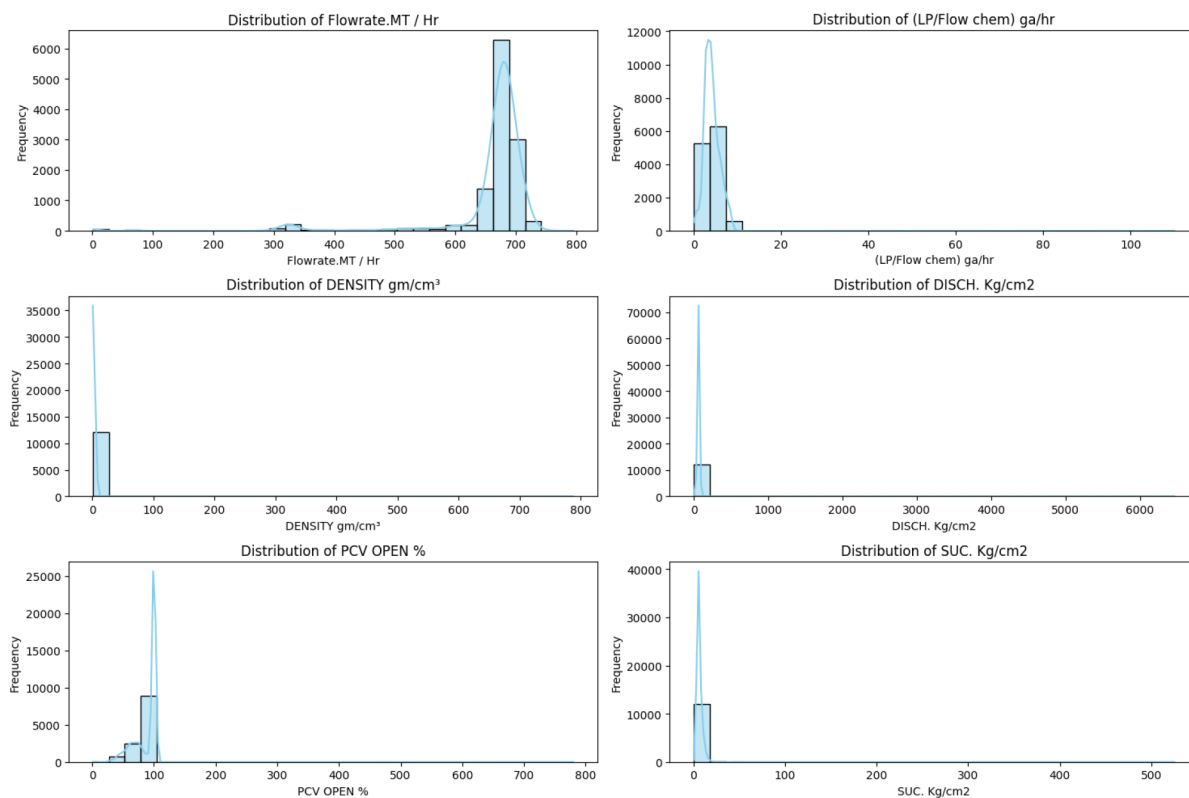


Figure 4—Density Plot of Process Variables

Outlier Validation and Removal

Once statistical outliers were identified, they were reviewed with process engineers to differentiate true abnormal operational conditions from erroneous data. Only data errors were removed, ensuring that meaningful extreme operational cases were preserved. Approved outliers were eliminated using the IQR-based filtering method, resulting in a cleaned dataset (Figure 5 and 6)

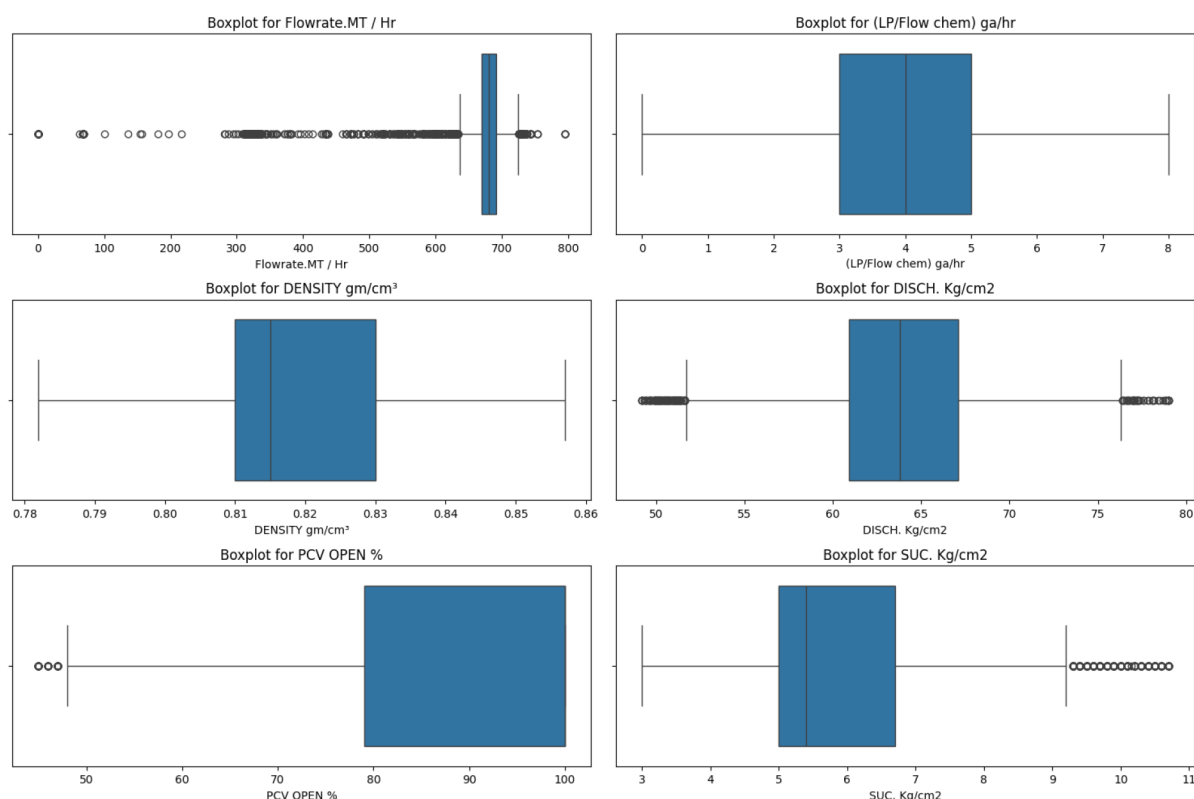


Figure 5—Box Plot of Cleaned Process Variables

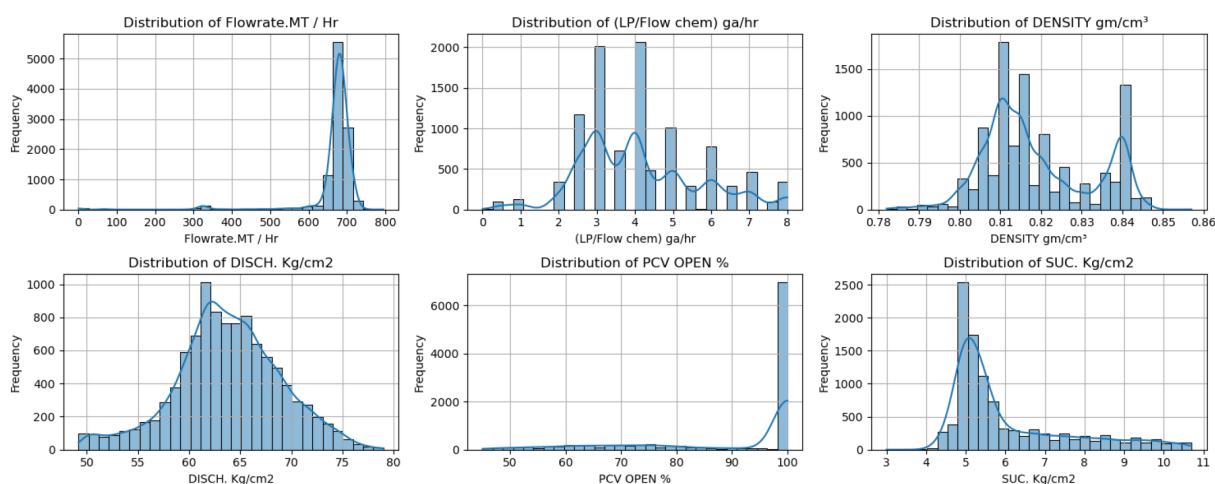


Figure 6—Cleaned Data Distribution after outlier removal

Feature Scaling and Encoding

To ensure uniform influence of each feature in dataset, standard scaling was applied to all continuous variables (e.g., flowrate, pressures, and crude density). The data were normalized to a zero mean and unit

variance, preventing variables with larger magnitudes (such as pressures in $\text{kg}\cdot\text{cm}^{-2}$) from dominating smaller-scale features during model training.

In addition, categorical variables such as pumping configurations and station identifiers were label-encoded to convert them into numerical values suitable for machine learning algorithms. Label encoding ensures that the model can interpret categorical data without introducing unnecessary dimensionality, which would otherwise occur if one-hot encoding were used on high-cardinality variables.

Model Training and Testing

Five supervised regression models were evaluated in this study to predict the optimized DRA injection rate: Decision Tree (DT), Random Forest (RF), Support Vector Regressor (SVR), K-Nearest Neighbors (KNN), and Extreme Gradient Boosting (XGBoost). These models were selected because they represent diverse learning strategies tree-based, kernel-based, distance-based, and boosting approaches—allowing for a comprehensive comparison.

The dataset was divided into 80% training and 20% testing, ensuring sufficient data for learning while preserving unseen data for performance evaluation. All models were trained on the log-transformed target variable ($\log(\text{DRA rate})$) to stabilize variance and improve prediction accuracy. To optimize the hyperparameters for each model, a GridSearchCV strategy incorporating 10-fold cross-validation was employed. This method systematically evaluates different parameter combinations by dividing the dataset into ten subsets, training on nine, and validating on the remaining one. By repeating this process across all folds, the approach effectively reduces the risk of overfitting and ensures the selection of the most robust hyperparameter set for improved model generalization.

Decision Tree

The Decision Tree Regressor predicts the target variable y (DRA consumption in gal/hr) by repeatedly dividing the feature set into distinct segments that minimize the variance within each region. At each split, the algorithm selects the feature j and threshold t that minimize the Mean Squared Error (MSE) impurity function can see in [equation 1](#) ([Scikit-learn Developers, 2025](#)).

$$MSE = \frac{1}{N_m} \sum_{i \in R_m} (y_i - \widehat{y}_{R_m})^2 \quad [1]$$

where:

N_m is the number of samples in node m

R_m is the set of samples in node

$\widehat{y}_{R_m} = \frac{1}{N_m} \sum_{i \in R_m} y_i$ is the mean target value in region R_m

The optimal split at each node is determined by [Equation 2](#)

$$\min_{j,t} \left[\frac{N_{m_{\text{left}}}}{N_m} MSE_{m_{\text{left}}} + \frac{N_{m_{\text{right}}}}{N_m} MSE_{m_{\text{right}}} \right] \quad [2]$$

where m_{left} and m_{right} denote the child nodes resulting from the split.

The model hyperparameters such as the maximum depth allowed for each tree, minimum samples required to split a node and minimum samples per leaf were optimized using GridSearchCV with 10-fold cross-validation to prevent overfitting and improve generalization.

Random Forest

The Random Forest model, an ensemble of decision trees, was used to reduce variance and overfitting by averaging predictions from multiple trees.

Let $\{T_b\}_{b=1}^B$ be the set of B decision trees trained on bootstrap samples of the data.

For regression, the prediction for an input x is defined as in [equation 3 \(Wikipedia contributor, 2025\)](#)

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad [3]$$

where $T_b(x)$ represents the prediction from the b^{th} decision tree, and B corresponds to the hyperparameter $n_{\text{estimators}}$. This averaging process reduces the model's variance and improves generalization.

The model's key hyperparameters such as the total number of trees in the ensemble, the maximum depth allowed for each tree, and whether bootstrapping was enabled, were fine-tuned using a GridSearchCV approach combined with 10-fold cross-validation to enhance generalization capability.

Support Vector Regressor

Support Vector Machines (SVMs) are supervised learning algorithms commonly applied to tasks such as classification, regression, and the identification of outliers. It can be defined as shown in [Equation 4](#)

$$f(x) = w^T x + b \quad [4]$$

Where:

- x is the input feature vector,
- w is the weight vector (to be learned),
- b is the bias term (intercept),
- The goal is to keep $f(x)$ as close as possible to the actual target value y , within an acceptable margin ϵ .

The kernel function used in [equation 5 \[1\]](#)

$$\text{linear Kernel Function} = \langle x, x' \rangle \quad [5]$$

Support Vector Regression (SVR) models non-linear relationships without explicitly computing $\phi(x)$. The epsilon (ϵ) parameter sets a margin where errors are ignored, controlling model flexibility. In this study, hyperparameters including the regularization parameter C , kernel type, kernel coefficient γ , and epsilon ϵ were optimized using 10-fold cross-validation to improve prediction accuracy.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based learning algorithm that predicts the target value of a query point by averaging (for regression) or taking a majority vote (for classification) from its k nearest neighbors in the training dataset. The closeness of neighbors is determined by using a distance metric, with common choices including those defined in [Equations \(6\), \(7\), and \(8\)](#), respectively ([geeksforgeeks, 2025](#)):

Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad [6]$$

Manhattan distance:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad [7]$$

Minkowski Distance:

$$d(x, y) = \sum_{i=1}^n ((x_i - y_i)^p)^{\frac{1}{p}} \quad [8]$$

Here, n is the number of features, and p in the Minkowski distance allows flexibility when $p=2$, it becomes Euclidean distance, and when $p=1$, it becomes Manhattan distance.

The model's hyperparameters number of neighbors (n neighbors) weighting scheme (uniform or distance), and distance metric were optimized using cross-validation to balance bias and variance, improving predictive performance.

Extreme Gradient Boosting (XGBoost)

XGBoost is an advanced gradient boosting framework designed for high efficiency, scalability, and superior performance, particularly on structured or tabular datasets. It operates by building an additive ensemble of decision trees in a sequential manner, where each new tree is trained to correct the residual errors of the combined previous trees. XGBoost integrates regularization directly into its objective function. This approach helps prevent the model from fitting too closely to the training data and improves its performance on new, unseen data. Additional advantages include handling of missing values, parallel processing, and flexibility in model tuning.

The learning process can be viewed as an iterative procedure starting from an initial prediction (often set to zero or a constant value). At each iteration, a new decision tree is added to minimize the objective function, thereby improving predictions. Mathematically, the prediction for the i -th data point can be expressed in [equation 9](#) ([geeksforgeek, 2025](#)):

$$y_i = \sum_{k=1}^K f_k(x_i) \quad [9]$$

where:

- $\hat{y}_i$ is the predicted value for the i -th observation,
- K is the total number of trees in the ensemble,
- $f_k(x_i)$ denotes the output of the k -th decision tree for the input x_i .

The objective function optimized in XGBoost is defined in [equation 10](#):

$$L(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k) \quad [10]$$

where:

- $l(\hat{y}_i, y_i)$ is the loss function measuring the difference between predicted and true values,
- $\Omega(f_k)$ is the regularization term penalizing model complexity to prevent overfitting,

ϕ represents all parameters in the model.

In this study, a two-stage hyperparameter tuning approach was used to optimize performance given the large parameter search space. RandomizedSearchCV was first applied to broadly explore possible configurations, followed by GridSearchCV to fine-tune within the most promising parameter ranges. Key hyperparameters adjusted included:

n_estimators: number of boosting rounds,

max_depth: maximum tree depth,

learning_rate: step size controlling each tree's contribution,

subsample: fraction of training samples used per boosting round.

This approach balanced bias and variance, resulting in a model with high predictive accuracy and robust generalization.

Models Performance Evaluation

The effectiveness of each regression model was evaluated based on three primary criteria as follows:

- R-squared (R^2)
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)

Together, these above measures offer a comprehensive understanding of the models prediction quality and error rate.

R^2 (Coefficient of Determination) measures the proportion of variance in the target variable that can be explained by the model. It is calculated as explain in [equation 11 \(greeksforgreeks, 2025b\)](#).

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad [11]$$

where SS_{res} is the residual sum of squares and SS_{tot} is the total sum of squares. Higher values of R^2 indicate a better model fit.

RMSE (Root Mean Square Error) evaluates the standard deviation of the residuals, penalizing larger prediction errors more heavily. It is given by [equation 12 \(greeksforgreeks, 2025c\)](#):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad [12]$$

where,

y_i = actual value

$\hat{y}_i$ = predicted value

n = number of observations

RMSE was used as the primary optimization objective during cross-validation because it is sensitive to large deviations.

Mean Absolute Error (MAE) quantifies the average size of the prediction errors, treating all deviations equally without considering their direction. As a single numerical value, it offers an intuitive measure of how close the predictions are to the actual target values. It is calculated as expressed in [equation 13 \(greeksforgreeks, 2025a\)](#):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad [13]$$

Were,

y_i : Actual value for the i th observation

$\hat{y}_i$: Calculated value for the i th observation

n : Total number of observations

To ensure robust performance estimation, the mean RMSE across 10-fold cross-validation (CV RMSE) was used as the primary validation metric. Final model performance was then evaluated on the held out 20% test dataset to assess generalization capability. For interpretability, both predictions and actual target values were inverse transformed to revert to the original DRA injection rate scale

Results and Observations

To understand the relationships among key operational variables, a pairwise scatter plot matrix (Figure X) was created for Crude Rate, DRA Rate, Density, Disch Pressure, PCV Opening (%), and Suction Pressure. This visualization presents both the individual feature distributions along the diagonal and the interactions between pairs of variables in the off-diagonal scatter plots.

The scatter plots reveal nonlinear and regime-dependent patterns across several variable pairs. For instance, the relationship between Crude Rate and DRA Rate exhibits distinct clusters, likely corresponding to different pump configurations, while the PCV Opening (%) versus Disch Pressure plot displays banding

patterns that reflect control valve step positions. These patterns indicate that the underlying data dynamics are complex and cannot be effectively modeled using simple linear approaches.

Moreover, there is no visual indication of strong multicollinearity among features; the variables display independent variation with only moderate correlations, suggesting that each feature contributes unique information.

The presence of curved clusters and nonlinear dependencies in the data motivated the selection of nonlinear machine learning models such as Random Forest, XGBoost, and Support Vector Regression (SVR) with RBF kernels. These models are better suited to capture the intricate interactions and multi-regime behavior of the operational parameters, offering improved predictive accuracy compared to linear regression techniques.

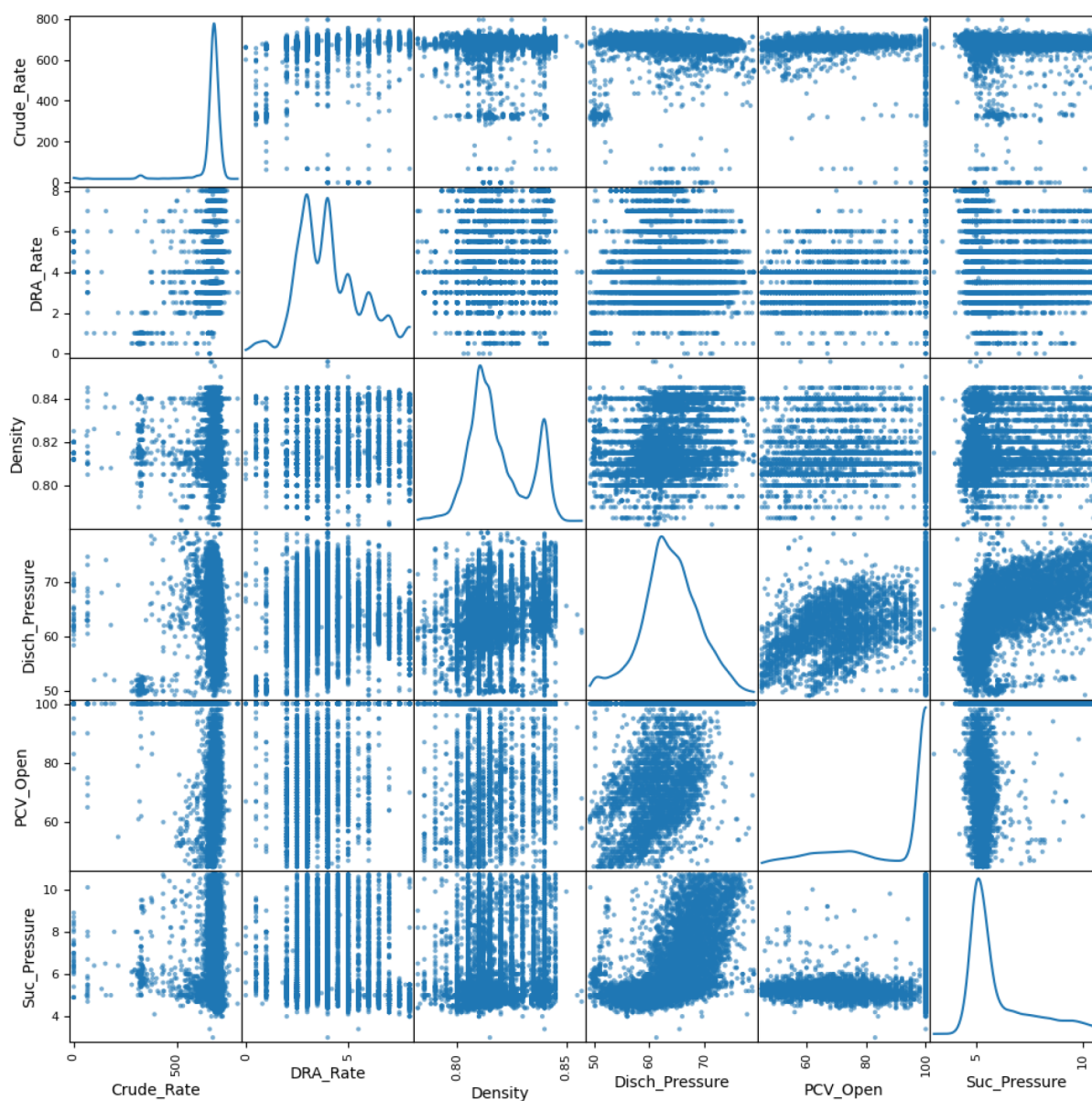


Figure 7—Pari plots between Six parameters: Density (D), Crude Flowrate, DRA Injection Rate, Discharge Pressure, Suction Pressure, PCV Opening (%)

Performance of Decision Tree Regressor

The performance of six Decision Tree regression models with different hyperparameter settings was evaluated on the test dataset. The results, summarized below [Table 1](#), reveal how tuning the tree depth and minimum sample parameters affect predictive accuracy.

Table 1—Performance metrics (Test R², MAE, and RMSE) for Decision Tree Regressor Models with different hyperparameter settings evaluated on test data.

Model	Hyperparameters	Test R ²	Test MAE	Test RMSE
1	max_depth=3, min_samples_split=2, min_samples_leaf=1	0.6469	0.5423	0.6751
2	max_depth=5, min_samples_split=2, min_samples_leaf=2	0.6635	0.5145	0.6590
3	max_depth=10, min_samples_split=5, min_samples_leaf=4	0.7086	0.4463	0.6133
4	max_depth=None, min_samples_split=2, min_samples_leaf=1	0.5888	0.4120	0.7285
5	max_depth=20, min_samples_split=10, min_samples_leaf=4	0.6837	0.4371	0.6389
6	max_depth=15, min_samples_split=5, min_samples_leaf=2	0.6461	0.4363	0.6759

Model 3, configured with a max_depth of 10, min_samples_split of 5, and min_samples_leaf of 4, delivered the best overall performance. This model achieved the highest test R² of 0.71, indicating it explained 71% of the variance in the target variable. It also had the lowest RMSE (0.61) and MAE (0.45), suggesting it produced the most precise and reliable predictions.

Models with shallower trees, such as Model 1 (max_depth=3), showed moderate performance with an R² of 0.65, but higher RMSE (0.68) and MAE (0.54), indicating underfitting due to limited model complexity.

On the other hand, Model 4, which had no restriction on tree depth (max_depth=None), recorded a lower R² of 0.59, despite achieving the lowest MAE (0.41). This discrepancy suggests that while it made some very accurate predictions, overall it overfitted the training data and generalized poorly on the test set, leading to a higher RMSE (0.73).

Intermediate depth models such as Model 5 (max_depth=20) and Decision Tree #6 (max_depth=15) performed reasonably well with R² values around 0.68 and 0.65, respectively, and comparable RMSE and MAE values, reflecting a balance between under- and overfitting.

Increasing the minimum number of data points required to divide a node, along with raising the least number of samples allowed in each leaf, contributed to reducing overfitting and improving generalization by simplifying the model structure and avoiding unnecessarily complex splits.

Performance of Random Forest Regressor

The performance of the Random Forest regression models was systematically evaluated across six hyperparameter configurations, revealing clear trends in predictive accuracy and model complexity. The configuration with 200 estimators and no maximum depth constraint achieved superior results, attaining a test R² of 0.80, alongside the lowest MAE (0.38) and RMSE (0.51). This indicates the model's exceptional capability to capture complex nonlinear relationships inherent in the data.

Models with limited tree depth or fewer estimators exhibited reduced performance, with R² values near 0.68 and correspondingly higher error metrics, reflecting underfitting due to insufficient model complexity. The introduction of regularization parameters such as minimum samples per split and leaf size in intermediate configurations provided a valuable trade-off.

These intermediate configurations slightly lowered R^2 to the range of 0.75–0.79 while maintaining robust predictive stability, as evidenced by moderate MAE (0.39–0.43) and RMSE (0.52–0.56) values. These findings, summarized in Table 2, emphasize the critical balance between ensemble size, tree depth, and regularization in optimizing Random Forest models for accurate and generalizable predictions in complex operational datasets.

Table 2—Performance metrics (Test R^2 , MAE, and RMSE) for Random Forest Regressor Models with different hyperparameter

Model	Hyperparameters	Test R^2	Test MAE	Test RMSE
1	n_estimators=50, maxdepth=5	0.6827	0.5139	0.6399
2	n_estimators=100, max_depth=10	0.7782	0.4152	0.5350
3	n_estimators=200, max_depth=None	0.7969	0.3761	0.5120
4	n_estimators=100, max_depth=20, min_samples_split=5	0.7891	0.3870	0.5217
5	n_estimators=150, max_depth=15, min_samples_leaf=2	0.7877	0.3911	0.5235
6	n_estimators=200, max_depth=10, min_samples_split=10, min_samples_leaf=4	0.7537	0.4333	0.5638

Performance of XG Boost

The XGBoost regression models demonstrated strong predictive capability for estimating the required DRA injection rate, as indicated by high R^2 values and relatively low MAE and RMSE scores across most hyperparameter configurations (Table 3). These metrics highlight the algorithm's ability to capture complex nonlinear relationships between operational inputs and the target output. Furthermore, the overall stability in performance across multiple configurations suggests a robust modelling framework well-suited for this prediction task.

Table 3—Performance metrics (Test R^2 , MAE, and RMSE) XG Boost Models with different hyperparameter settings evaluated on test data

Model	Hyperparameters	Test R^2	Test MAE	Test RMSE
1	n_estimators=100, max_depth=3, learning_rate=0.1, subsample=1.0	0.6986	0.4985	0.6237
2	n_estimators=100, max_depth=5, learning_rate=0.05, subsample=0.8	0.7306	0.4666	0.5896
3	n_estimators=200, max_depth=3, learning_rate=0.1, subsample=1.0	0.7173	0.4765	0.6041
4	n_estimators=200, max_depth=7, learning_rate=0.01, subsample=1.0	0.5627	0.5749	0.7512
5	n_estimators=150, max_depth=5, learning_rate=0.1, subsample=0.9	0.7668	0.4222	0.5486
6	n_estimators=250, max_depth=4, learning_rate=0.05, subsample=0.8	0.7444	0.4527	0.5744

Among the tested configurations, Model 5 achieved the best performance, with the highest Test R^2 of 0.7668, indicating that approximately 76.7% of the variance in the DRA injection rate was explained by the model inputs. It also recorded the lowest Test MAE (0.4222) and RMSE (0.5486), reflecting a high level of predictive precision with minimal deviation from actual values. These results demonstrate that this combination of moderate tree depth (5), sufficient ensemble size (150 trees), and balanced learning rate (0.1) effectively captures data complexity while avoiding overfitting.

In comparison, Model 4 exhibited the weakest performance, with an R^2 of only 0.5627 and the highest error metrics. This underperformance can be attributed to the combination of a deep tree structure (max_depth=7) with an excessively low learning rate (0.01), which likely limited convergence efficiency despite increased complexity. The remaining configurations (Models 1, 2, 3, and 6) yielded R^2 scores in the range of 0.6986–0.7444, with moderate error values, indicating reasonable accuracy but slightly reduced generalization compared to the optimal setup.

Overall, the results confirm XGBoost robustness for this prediction task while emphasizing the importance of careful hyperparameter tuning. Balancing learning rate, tree depth, and number of estimators emerges as the key to maximizing predictive performance and ensuring model generalization across unseen operational data.

Performance of Support Vector Machine

The Support Vector Regression (SVR) models exhibited varying degrees of predictive effectiveness for estimating the required DRA injection rate, with performance highly sensitive to the choice of kernel function and hyperparameters (Table 4). The results reveal that the ‘rbf’ kernel consistently outperforms other kernel types, indicating its superior ability to model the nonlinear relationships present in the operational dataset.

Table 4—Performance metrics (Test R², MAE, and RMSE) Support Vector Machine Models with different hyperparameter settings evaluated on test data.

Model	Hyperparameters	Test R ²	Test MAE	Test RMSE
1	kernel=rbf. C=150, epsilon=0.1	0.6667	0.4494	0.6559
2	kernel=rbf. C=10.0, epsilon=0.1	0.6987	0.4543	0.6235
3	kernel=linear, C=1.0, epsilon=0.1	0.4059	0.6670	0.8756
4	kernel=linear. C=10.0, epsilon=0.2	0.4080	0.6729	0.8741
5	kernel=poly. degree=2, C=1.0, epsilon=0.1	0.4156	0.6324	0.8684
6	kernel=poly, degree=3, C=5.0, epsilon=0.05	0.1938	0.6427	1.2412

Among the tested configurations, Model 2 employing an ‘rbf’ kernel with C=10.0 and $\epsilon=0.1$ —achieved the highest Test R² of 0.6987. This indicates that nearly 70% of the variance in DRA injection rates can be explained by its predictors. The model also produced a competitive MAE of 0.4543 and an RMSE of 0.6235, suggesting that its predictions are relatively close to actual operational values. Model 1, which also used an ‘rbf’ kernel but with a higher C value (150), achieved slightly lower accuracy, indicating that increasing C did not necessarily improve generalization.

In contrast, models employing ‘linear’ and ‘poly’ kernels (Models 3, 4, 5, and 6) demonstrated notably weaker performance. These configurations produced substantially lower R² values, with Model 6 even returning a negative R² score, meaning it performed worse than a baseline model that simply predicts the mean. Error metrics for these models were also significantly higher, confirming their inability to capture the complex nonlinear patterns in the dataset.

Overall, these findings highlight the critical importance of kernel selection in SVR modeling for this application. The ‘rbf’ kernel emerges as the most effective option for capturing the underlying nonlinear relationships governing DRA injection rate prediction, while ‘linear’ and ‘poly’ kernels are less suitable in this operational context.

Performance of K-Nearest Neighbors (KNN)

The K-Nearest Neighbors (KNN) approach showed solid performance in predicting the necessary DRA injection rate. Its effectiveness was significantly impacted by the selection of hyperparameters such as the number of neighbors, the method of assigning weights, and the distance calculation metric, as shown in Table 5. The findings suggest that, with proper tuning, KNN is capable of identifying underlying patterns within the operational data and delivering predictive accuracy comparable to other models assessed in this research.

Table 5—Performance metrics (Test R², MAE, and RMSE) Support Vector Machine Models with different hyperparameter settings evaluated on test data

Model	Hyperparameters	Test R ²	Test MAE	Test RMSE
1	n_neighbors=3, weights=uniform, metric=euclidean	0.7370	0.3925	0.5825
2	n_neighbors=5, weights=uniform, metric=manhattan	0.7560	0.3964	0.5612
3	n_neighbors=7, weights=distance, metric=minkowski	0.7588	0.3943	0.5579
4	n_neighbors=10, weights=distance, metric=euclidean	0.7420	0.4169	0.5771
5	n_neighbors=8, weights=uniform, metric=minkowski	0.7180	0.4439	0.6033
6	n_neighbors=6, weights=distance, metric=manhattan	0.7771	0.3706	0.5364

Out of all the configurations evaluated, Model 6 delivered the strongest performance among the KNN models. It reached a Test R² of 0.7771, indicating that nearly 77.7% of the variation in DRA injection rates was captured using the selected input variables. This model also produced the smallest Test MAE 0.3706 and RMSE 0.5364, reflecting high prediction accuracy with values closely aligned to real operational data. The best results were obtained when using 6 nearby data points, a distance-based weighting scheme, and the Manhattan distance function, highlighting that considering proximity in weighting can greatly enhance model precision.

Several other configurations also performed competitively. For example, Models 2 and 3 produced R² values of 0.7560 and 0.7588, respectively, with relatively low MAE and RMSE values, although they did not outperform Model 6 across all metrics. The consistency of high R² scores across multiple configurations confirms that KNN is a viable approach for this prediction task, provided the parameters are well-optimized to enhance predictive capability.

Models Performance Comparison Analysis

A comparative evaluation of all optimized machine learning models Decision Tree, Random Forest, XGBoost, Support Vector Regression (SVR), and K-Nearest Neighbors (KNN) was conducted to determine the most effective approach for predicting the required DRA injection rate. The performance metrics, including Test R², MAE, and RMSE, for each model are summarized in [Table 6](#).

Table 6—Comparative performance analysis of various machine learning models with their respective hyperparameters, evaluated using Test R², MAE, and RMSE metrics.

Model	Hyperparameters	Test R ²	Test MAE	Test RMSE
Decision Tree	max_depth=10, min_samples_split=5, min_samples_leaf=4	0.7086	0.4463	0.6133
Random Forest	n_estimators=200, max_depth=None	0.7969	0.3761	0.5120
XG Boost	n_estimators=150, max_depth=5, learning_rate=0.1, subsample=0.9	0.7668	0.4222	0.5486
SVR	kernel=rbf, 010.0, epsilon=0.1	0.6987	0.4543	0.6235
KNN	n_neighbors=6, weights=distance, metric=manhattan	0.7771	0.3706	0.5364
Decision Tree	max_depth=10, min_samples_split=5, min_samples_leaf=4	0.7086	0.4463	0.6133

Among all evaluated models, Random Forest demonstrated the highest overall performance. It achieved the highest R² value of 0.7969, explaining nearly 79.7% of the variance in DRA injection rates, indicating strong predictive capability and an ability to capture complex nonlinear relationships within the dataset. Additionally, Random Forest recorded the lowest MAE (0.3761) and RMSE (0.5120), confirming both high accuracy and minimal prediction error, even when penalizing larger deviations more heavily.

While KNN and XGBoost also achieved competitive results—KNN producing an R^2 of 0.7771 with the lowest MAE (0.3706) among all models, and XGBoost yielding an R^2 of 0.7668 with balanced error metrics Random Forest maintained superior consistency across all three evaluation criteria. This consistent performance advantage suggests that Random Forest offers an optimal balance between model complexity, generalization capability, and predictive precision.

The Decision Tree model, although less complex, achieved moderate accuracy ($R^2 = 0.7086$) but exhibited higher error metrics compared to ensemble approaches, illustrating the benefits of aggregation methods such as Random Forest in reducing variance and improving generalization. Similarly, SVR with an ‘rbf’ kernel ($R^2 = 0.6987$) provided reasonable performance but lagged behind ensemble and instance-based methods, highlighting the importance of model architecture in addressing the nonlinearities present in the operational data.

Overall, the results underscore that while multiple algorithms can provide acceptable accuracy when properly tuned, Random Forest consistently delivers the best combination of variance explanation, prediction accuracy, and error minimization for DRA injection rate prediction in this study.

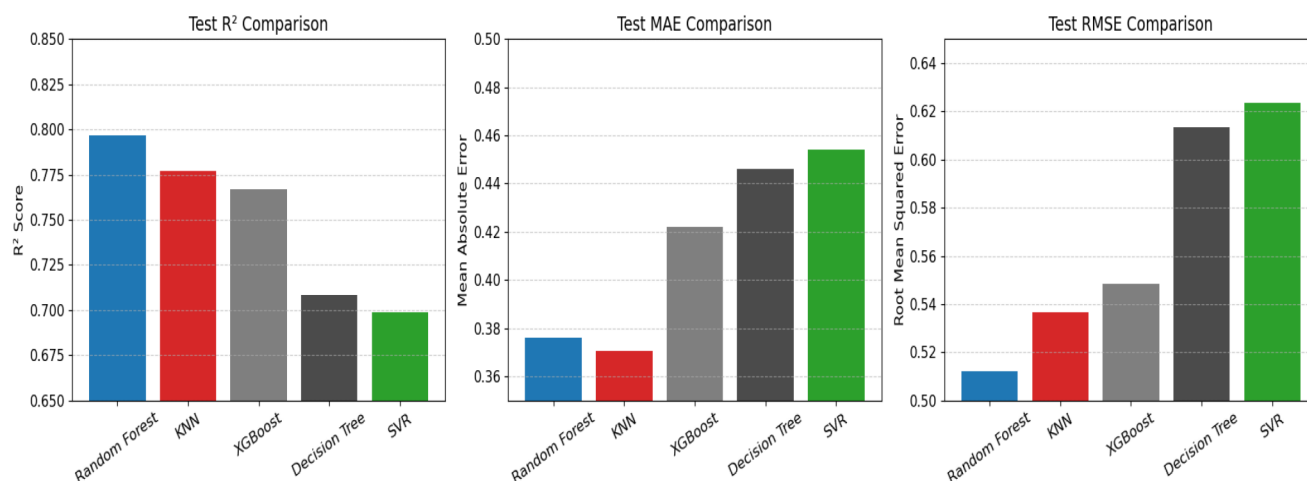


Figure 8—Comparison of model performance using Test R^2 , MAE, and RMSE metrics for Random Forest, KNN, XGBoost, Decision Tree, and SVR

Field Trial: Model Validation and Practical Cost Savings

To evaluate the practical applicability of the developed machine learning model, a live field trial was conducted across four pumping stations (PS-1 to PS-4) in a one-day operational test. The objective was to compare the actual Drag Reducing Agent (DRA) consumption with the model's predicted optimal DRA injection rate, under real pipeline conditions.

As illustrated in Figure 9, the total actual DRA consumption across all stations amounted to 400 US gallons, whereas the model recommended a total of 376 US gallons, yielding a net saving of 24 US gallons (6% reduction) for the same throughput. Station-level results showed consistent efficiency, with the predicted consumption remaining lower than or closely aligned with actual values at each point. Notably:

- At PS-1, the model suggested a slightly higher dosage (117 vs 113 USG), likely adjusting for local flow variability.
- At PS-2, PS-3, and PS-4, the model predicted lower DRA consumption without compromising flow performance.

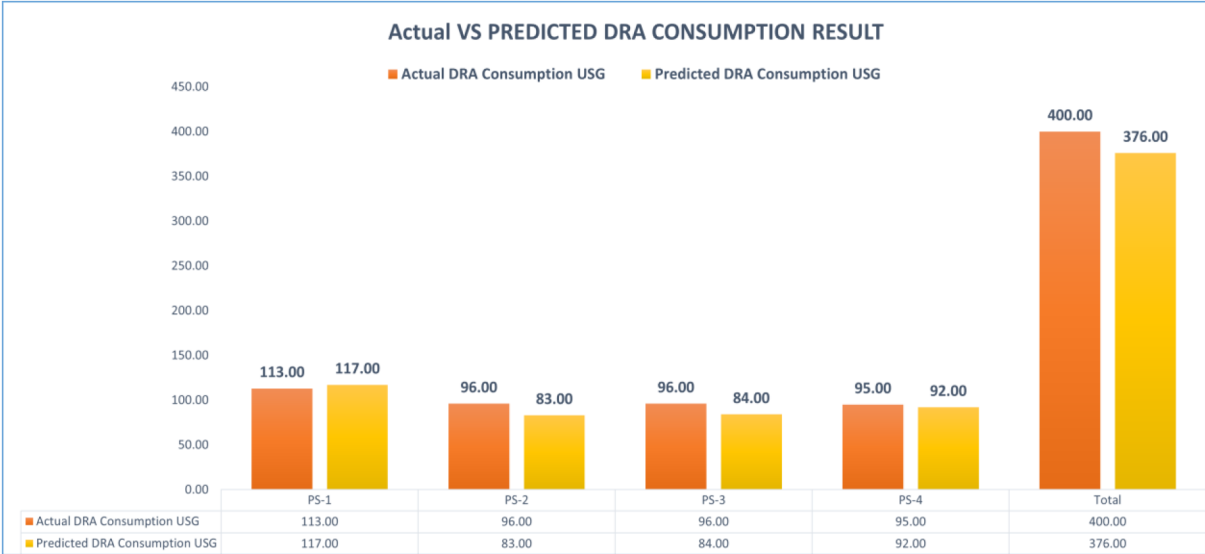


Figure 9—Comparison between observed and predicted outputs from the Random Forest model for four pumping stations using real-time operational data collected during a one-day field test.

This trial demonstrates the model's potential to optimize chemical usage, reduce operational costs, and support data-driven decision-making. Assuming average DRA cost of X USD per gallon, this translates to a direct daily saving of $X \times 24 = Y$ USD, which, when projected annually, represents a significant financial impact. Moreover, reducing chemical dosage also contributes to environmental sustainability and operational efficiency without sacrificing throughput performance.

Conclusion

This study demonstrated the effectiveness of an AI-driven framework for optimizing Drag Reducing Agent (DRA) injection rates in crude oil pipeline operations. By leveraging historical operational data and applying multiple supervised learning models, the framework identified Random Forest regression as the most accurate and robust predictor, achieving an R^2 of 0.80 and the lowest error metrics among all tested models. The approach successfully addressed the limitations of static, rule-based dosing strategies by providing adaptive, data-driven recommendations that respond to dynamic operating conditions.

The live field trial validated the practical benefits of the proposed system, achieving a 6% reduction in DRA consumption without compromising throughput. This outcome translates into measurable cost savings and reduced environmental impact, highlighting the potential of machine learning to support both economic and sustainability objectives in the oil and gas sector.

The developed methodology is scalable and can be extended to other chemical dosing and pipeline optimization scenarios, offering a pathway toward more intelligent, automated, and sustainable pipeline management. Future work may explore real-time model integration with SCADA systems, incorporation of broader operational datasets, and continuous model retraining to further enhance predictive accuracy and adaptability under evolving operating conditions.

References

Berge, B. K. S. O. (1996). *Increased pipeline throughput using drag reducer additives (DRA): Field experiences*. 203–208.

Burger, E. D.; M. W. R.; W. H. A. (1982). *Flow increase in trans Alaska pipeline through use of polymeric drag-reducing additive*. 377–386.

geeksforgeek. (2025, July 23). *XGBoost*.

geeksforgeeks. (2025, July 23). *k-Nearest Neighbours*.

greksforgreeks. (2025a, May 27). *How to Calculate Mean Absolute Error in Python?*

greksforgreeks. (2025b, July 11). *R-squared in Regression Analysis in Machine Learning*.

- greeksgreek. (2025c, July 15). *Root Mean Square Error in R Programming*.
- Jubran, B. A., Zurigat, Y. H., & Goosen, M. F. A. (2005). Drag reducing agents in multiphase flow pipelines: Recent trends and future needs. *Petroleum Science and Technology*, **23**(11–12), 1403–1424. <https://doi.org/10.1081/LFT-200038223>
- Modisette, J. P. (n.d.). *Dynamic Programming Applied to DRA Optimization and Power Contract Optimization*.
- Scikit-learn Developers. (2025). *Decision Trees*.
- Wikipedia contributor. (2025). *Random forest*.
- S.-l. Developers, "Support Vector Machines (SVM) - scikit-learn documentation," 2025.